

UNIVERZITET U NOVOM SADU

FAKULTET TEHNIČKIH NAUKA

Predmet: Primenjeni algoritmi u upravljačkim sistemima

Practical Byzantine Fault Tolerance (PBFT)

Seminarski rad

Sadržaj

1. Uvod i motivacija problema konsenzusa
2. Opis algoritma PBFT
 - 2.1. Sistemski model i pretpostavke
 - 2.2. Osnovni pojmovi
3. Glavne faze algoritma
 - 3.1. Klijentska interakcija
 - 3.2. Normalan rad — tri faze protokola
 - 3.3. Promena view-a (view-change)
4. Simulacija grešaka
5. Arhitektura implementiranog simulatora
 - 5.1. Tehnologija i model konkurentnosti
 - 5.2. Slojevita infrastruktura
 - 5.3. Struktura PBFT paketa
 - 5.4. Pojednostavljenja u odnosu na izvorni rad
6. Demonstracioni scenariji i verifikacija ispravnosti
7. Zaključak

1. Uvod i motivacija problema konsenzusa

Savremeni distribuirani sistemi – od bankarskih platformi i baza podataka u oblaku, do blockchain mreža – sve više zavise od replikacije podataka na više nezavisnih čvorova kako bi se obezbedila visoka dostupnost i otpornost na kvarove. Osnovni izazov u takvim sistemima jeste problem konsenzusa: kako obezbediti da svi ispravni čvorovi u sistemu dođu do istog, dogovorenog sleda operacija, uprkos tome što neki čvorovi mogu da otkazu, kasne ili se ponašaju na neočekivan način.

Klasična literatura o distribuiranim sistemima najčešće razmatra takozvane benigne (fail-stop) greške, gde čvor koji otkaze jednostavno prestaje da radi ili propušta korake, ali nikada ne šalje netačne ili zlonamerne poruke. Algoritmi poput Paxos-a (Lamport) rešavaju problem konsenzusa upravo pod ovom pretpostavkom. Međutim, u realnim sistemima uzroci otkaza su mnogo raznovrsniji: greške u softveru, propusti operatera, hardverski kvarovi i, sve češće, namerni zlonamerni napadi. Sve ove situacije mogu dovesti do proizvoljnog (Byzantine) ponašanja čvora – čvor može slati različite, međusobno kontradiktorne poruke različitim učesnicima, kasniti selektivno, ili se pretvarati da je ispravan dok potkopava ispravnost sistema.

Problem dogovaranja u prisustvu proizvoljno neispravnih učesnika prvi put je formalno postavljen kroz takozvani problem vizantijskih generala (Lamport, Shostak, Pease, 1982), koji pokazuje da je, u sistemu sa n učesnika, moguće postići pouzdan dogovor samo ako je broj neispravnih učesnika manji od trećine ukupnog broja. Iako je problem teorijski rešen još osamdesetih godina, dugo vremena predložena rešenja bila su prespora ili su zahtevala nerealne pretpostavke (npr. sinhronost mreže) da bi se koristila u praksi.

Prekretnicu predstavlja rad Miguel-a, Castro-a i Liskov, “Practical Byzantine Fault Tolerance and Proactive Recovery” (ACM Transactions on Computer Systems, 2002), koji opisuje algoritam PBFT – prvi algoritam replikacije stanja mašine koji je istovremeno: (1) bezbedan u potpuno asinhronim mrežama, bez oslanjanja na pretpostavke o vremenskim granicama, (2) dovoljno efikasan da se koristi u praksi, i (3) otporan na proizvoljno (byzantine) ponašanje do jedne trećine učesnika. PBFT predstavlja osnovu na kojoj počiva veliki broj savremenih sistema, uključujući i mnoge algoritme konsenzusa korišćene u blockchain tehnologijama.

Cilj ovog seminarskog rada je: opisati teorijske osnove i glavne faze algoritma PBFT onako kako su definisane u izvornom radu, i predstaviti arhitekturu i implementaciju simulatora ovog algoritma, urađenog u programskom jeziku Go, koji demonstrira ključna svojstva algoritma – bezbednost (safety) i živost (liveness) – kroz nekoliko konkretnih scenarija, uključujući normalan rad, pad primarnog čvora, zlonamerno (byzantine) ponašanje primarnog čvora i gubitak poruka usled nepouzdanosti mreže.

2. Opis algoritma PBFT

2.1 Sistemski model i pretpostavke

PBFT posmatra replicirani servis kao determinističku mašinu stanja (state machine) koja se izvršava identično na svakoj od N replika. Klijenti šalju zahteve za izvršavanje operacija; algoritam garantuje da sve ispravne replike izvršavaju iste operacije istim redosledom, čime se postiže svojstvo linearizabilnosti – sistem se, iz perspektive klijenta, ponaša kao jedna, centralizovana mašina.

Mreža koja povezuje replike smatra se nepouzdanom u asinhronom smislu: poruke mogu biti izgubljene, duplirane, kasniti proizvoljno dugo ili stići van redosleda slanja. Algoritam ne prati nikakvu pretpostavku o gornjoj granici kašnjenja poruka kada je u pitanju svojstvo bezbednosti (safety) – ono mora važiti bezuslovno, čak i pod aktivnim napadom uskraćivanja usluge. Za svojstvo živosti (liveness), odnosno garanciju da će klijentski zahtevi na kraju biti obrađeni, algoritam se oslanja na slabu pretpostavku da kašnjenja poruka na kraju prestaju neograničeno da rastu.

Ključna pretpostavka algoritma tiče se broja neispravnih replika: ako je f maksimalan broj replika koje mogu istovremeno biti neispravne (uključujući i zlonamerno ponašanje), potreban je ukupan broj replika:

$$N = 3f + 1$$

Razlog ove granice leži u argumentu preseka kvoruma: da bi se garantovala saglasnost, svaka odluka mora biti potvrđena od strane kvoruma od $2f+1$ replika. Ako postoje dva različita kvoruma iste veličine, njihov presek mora sadržati bar jednu zajedničku, ispravnu repliku – u suprotnom bi dva različita kvoruma mogla doneti dve različite, međusobno kontradiktorne odluke, a da nijedna ispravna replika to ne primeti. Matematički, presek dva skupa veličine $2f+1$ iz ukupnog skupa od N elemenata ima najmanje $2(2f+1) - N$ elemenata: da bi taj presek sadržao bar $f+1$ elemenata (dovoljno da garantuje bar jedan ispravan član, pošto je najviše f replika neispravno), mora važiti $N \geq 3f+1$.

2.2 Osnovni pojmovi

- View (pogled) – konfiguracija sistema u kojoj je tačno jedna replika određena za primary (lidera), a sve ostale su backup (prateće) replike. Views su numerisani uzastopnim celim brojevima počevši od nule.
- Primary – replika zadužena da dodeljuje redne brojeve (sequence numbers) klijentskim zahtevima. Bira se determinističkom formulom $p = v \bmod N$, gde je v broj trenutnog view-a – na taj način svaka replika, uključujući i neispravne, periodično dobija ulogu primary-a.

- Redni broj (sequence number) – jedinstven, rastući broj koji primary dodeljuje svakom zahtevu, čime se definiše ukupan redosled izvršavanja operacija.
- Digest – kriptografski sažetak (u radu MAC/hash) sadržaja poruke, koji omogućava da se brzo i pouzdano proverí da li dve poruke nose identičan sadržaj, bez potrebe da se prenosi ceo sadržaj.
- Jak kvorum (strong/prepared certificate) – skup od najmanje $2f+1$ međusobno saglasnih poruka različitih replika; garantuje da se bilo koja dva različita jaka kvoruma seku u bar jednoj ispravnoj replici.
- Slab kvorum (weak certificate) – skup od najmanje $f+1$ poruka; garantuje samo da je bar jedna od njih poslata od strane ispravne replike (koristi ga npr. klijent pri prihvatanju odgovora).

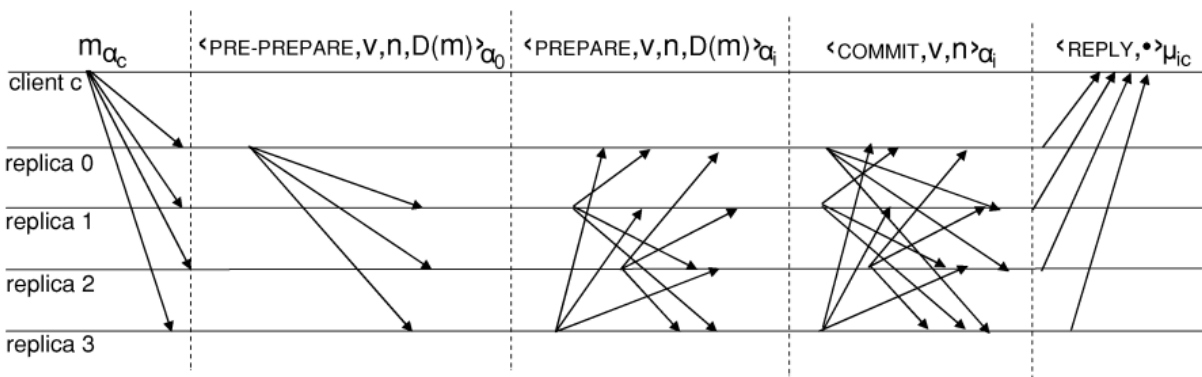
3. Glavne faze algoritma

3.1 Klijentska interakcija

Klijent koji želi da izvrši operaciju šalje zahtev (REQUEST) repliciranom servisu i zatim čeka odgovore. Pošto pojedinačno replika može biti neispravna i vratiti netačan rezultat, klijent ne prihvata odgovor od samo jedne replike – čeka da isti rezultat dobije od najmanje $f+1$ različitih replika (slab kvorum). Pošto je najviše f replika neispravno, ovakav skup podudarnih odgovora garantovano sadrži bar jedan odgovor ispravne replike, čime je rezultat pouzdan.

3.2 Normalan rad – tri faze protokola

Kada nema grešaka, obrada jednog klijentskog zahteva prolazi kroz tri faze, ilustrovane na Slici 1 originalnog rada (Castro & Liskov, 2002, Sekcija 4.3):



- Pre-prepare faza: primary, po prijemu zahteva, dodeljuje mu sledeći slobodan redni broj n u trenutnom view-u v i šalje svim backup replikama poruku $\text{PRE-PREPARE}(v, n, d)$, gde je d digest zahteva.

- Prepare faza: svaka backup replika koja prihvati PRE-PREPARE poruku (uz proveru da pripada trenutnom view-u i da ranije nije prihvatila drugačiji zahtev za isti par (v, n)) šalje svim ostalim replikama poruku PREPARE(v, n, d, i). Kada replika prikupi jak kvorum od $2f+1$ međusobno saglasnih poruka (PRE-PREPARE plus $2f$ odgovarajućih PREPARE poruka), kaže se da je zahtev pripremljen (prepared) na toj replici.
- Commit faza: čim je zahtev pripremljen, replika šalje poruku COMMIT(v, n, i) svim ostalim replikama. Kada prikupi jak kvorum od $2f+1$ COMMIT poruka, zahtev se smatra usvojenim (committed).

Nakon što je zahtev usvojen, replika ga izvršava – ali striktno redosledom rednih brojeva: zahtev sa rednim brojem n izvršava se tek pošto su izvršeni svi zahtevi sa manjim rednim brojevima. Ovime se garantuje da sve ispravne replike izvrše potpuno isti niz operacija istim redosledom, bez obzira na to kojim su redosledom poruke fizički stigle preko mreže. Nakon izvršenja, replika šalje odgovor (REPLY) direktno klijentu.

Razlog za postojanje dve odvojene faze (prepare i commit), umesto samo jedne, leži u potrebi da se saglasnost očuva i kroz eventualne promene view-a: prepare faza garantuje saglasnost unutar jednog view-a, dok commit faza garantuje da se ta saglasnost prenosi i kroz kasnije view-ove – bez commit faze, dve različite promene view-a mogle bi teorijski da usvoje dva različita zahteva za isti redni broj.

3.3 Promena view-a (view-change)

Ukoliko primary otkáže, kasni, ili se ponaša zlonamerno (npr. šalje različite PRE-PREPARE poruke različitim replikama za isti redni broj – takozvana equivocacija), backup replike to prepoznaju putem tajmera: ukoliko zahtev ne napreduje kroz faze protokola u razumnom vremenskom roku, replika posumnja da je primary neispravan.

U tom slučaju replika prelazi u neaktivno stanje trenutnog view-a i šalje svim ostalim replikama poruku VIEW-CHANGE($v+1$), tražeći prelazak na sledeći view. Ova poruka nosi i informaciju o poslednjem izvršenom rednom broju i eventualnom zahtevu na koji je replika još čekala odgovor. Kada replika koja treba da postane novi primary (određena formulom $p = (v+1) \bmod N$) prikupi jak kvorum od $2f+1$ VIEW-CHANGE poruka, ona zvanično preuzima ulogu primary-a u novom view-u, šalje svim replikama poruku NEW-VIEW kao dokaz legitimnosti prelaska, i ponovo predlaže (re-propose) svaki zahtev koji je ostao neizvršen iz prethodnog view-a.

Bitno je naglasiti da promena view-a utiče isključivo na svojstvo živosti sistema – sistem privremeno prestaje da napreduje dok se promena ne završi – dok svojstvo bezbednosti ostaje netaknuto u svakom trenutku: čak i ako neispravan primary pokuša da izazove kontradiktorne odluke, mehanizam jakog kvoruma sprečava da se bilo koja od dve sukobljene verzije zahteva ikada izvrši, sve dok se, kroz promenu view-a, ne uspostavi ispravno vodstvo.

4. Simulacija grešaka

Da bi se demonstrirala otpornost algoritma, implementirani simulator omogućava tri tipa grešaka nad replikama, u skladu sa zahtevima specifikacije projekta:

- Pad čvora (crash failure) – replika trajno prestaje da prima i šalje poruke, simulirajući fizički otkaz servera ili gubitak mrežne veze.
- Gubitak/kašnjenje poruka (omission failure) – pojedinačna replika sa određenom verovatnoćom ne prima poruke koje su joj upućene, simulirajući spor ili delimično nedostupan čvor, bez potpunog prekida rada.
- Vizantijsko (Byzantine) ponašanje – konkretno, equikovacija primary-a: neispravna replika, dok je u ulozi primary-a, namerno šalje dve različite verzije istog zahteva (različite digest vrednosti za isti par view/redni broj) dvema različitim grupama backup replika, pokušavajući da izazove nesaglasnost u sistemu.

Za svaki od ova tri tipa grešaka implementiran je poseban demonstracioni scenario (opisan u poglavlju 6), koji potvrđuje da algoritam zadržava ispravnost uprkos njihovom prisustvu, u granicama teorijske garancije $f = \lfloor (N-1)/3 \rfloor$.

5. Arhitektura implementiranog simulatora

5.1 Tehnologija i model konkurentnosti

Simulator je implementiran u programskom jeziku Go, koji poseduje ugrađenu podršku za konkurentno programiranje kroz lagane niti izvršavanja (gorutine) i kanale za komunikaciju među njima. Svaka replika i klijent predstavljaju zasebnu gorutinu koja se izvršava nezavisno i komunicira isključivo razmenom poruka preko kanala – čime se ispunjava zahtev specifikacije projekta da algoritam ne sme biti implementiran kao jedna sekvencijalna petlja, već kao skup nezavisnih, konkurentnih izvršnih jedinica.

5.2 Slojevita infrastruktura

Projekat se oslanja na zajedničku infrastrukturu (week03_04_parallel) koja obezbeđuje generičke mehanizme, nezavisne od PBFT algoritma:

- process – definiše osnovni interfejs Process koji svaka izvršna jedinica mora implementirati (metode ID() i Run()), kao i strukturu poruke koja putuje kroz sistem.
- link – slojevita implementacija mrežnog kanala: FairLossLink (osnovni, nepouzdan sloj sa mogućnošću gubitka poruka), StubbornLink (dodaje uporno ponovno slanje neisporučenih poruka), i PerfectLink (dodaje otkrivanje i odbacivanje duplikata).

- failures – modeli grešaka koji se ubacuju na nivou mreže: bez grešaka (NoFailure), pad čvora (CrashFailure), i gubitak poruka (OmissionFailure).
- runtime – orkestrator simulacije: pokreće gorutinu za svaki proces, rutira poruke između njih poštujući aktivni model grešaka, upravlja tajmerima za retransmisiju i neaktivnost, i beleži hronologiju svih mrežnih događaja.

Nad ovom infrastrukturom izgrađen je zaseban paket pbft, koji sadrži isključivo logiku samog algoritma i ne zavisi ni od jednog detalja njegove implementacije osim javnog interfejsa.

5.3 Struktura PBFT paketa

PBFT logika je organizovana u nekoliko tematski odvojenih datoteka radi preglednosti:

- types.go – definicija tipova poruka (PRE-PREPARE, PREPARE, COMMIT, REPLY, VIEW-CHANGE, NEW-VIEW) i osnovnih domenskih struktura (klijentski zahtev, digest funkcija).
- quorum.go – funkcije za izračunavanje maksimalnog broja neispravnih replika (f), veličine jakog i slabog kvoruma, i određivanje primary-a za dati view.
- broadcast.go – pomoćne funkcije za slanje poruka svim (ili odabranim) replikama, uključujući posebnu funkciju za simulaciju equivocacije.
- replica.go – centralna implementacija replike: obrada klijentskog zahteva, faze pre-prepare/prepare/commit, i izvršavanje zahteva striktno po redosledu rednih brojeva.
- viewchange.go – implementacija promene view-a: reakcija na istek tajmera, prikupljanje VIEW-CHANGE glasova, preuzimanje uloge primary-a i slanje NEW-VIEW poruke.
- byzantine.go – logika zlonamernog (equivocating) primary-a, korišćena isključivo u demonstracionom scenariju Byzantine ponašanja.
- client.go – implementacija klijenta koji šalje zahtev i čeka slab quorum podudarnih odgovora.
- recorder.go i checkers.go – mehanizam za beleženje toka izvršavanja (uključujući i hronološki, vremenski žigovan log događaja) i automatizovane provere ispravnosti nakon završetka simulacije.

5.4 Pojednostavljenja u odnosu na izvorni rad

U skladu sa dozvoljenim pojednostavljenjima navedenim u specifikaciji projekta, implementacije odstupa od potpune specifikacije iz originalnog rada na sledeće načine:

- Nisu implementirani mehanizmi periodičnog čuvanja stanja (checkpoints) i sabiranja otpadaka (garbage collection) opisani u Sekciji 4.4 originalnog rada.
- Nije implementiran mehanizam prenosa stanja (state transfer) niti proaktivni oporavak replika (proactive recovery) opisani u Sekcijama 5 i 6.2 originalnog rada.

- Promena view-a ne prenosi kompletne P i Q sertifikate iz originalnog rada, već samo jedan zahtev na koji se čekalo – dovoljno za demonstraciju osnovnog mehanizma, bez gubitka bezbednosne garancije u okviru testiranih scenarija.
- Digest poruka računa se kao skraćeni SHA-256 heš, umesto punog kriptografskog MAC mehanizma zasnovanog na simetričnim ključevima iz originalnog rada – integritet i redosled poruka na mrežnom nivou obezbeđuje sloj PerfectLink.

Nijedno od navedenih pojednostavljenja ne utiče na osnovnu logiku kvoruma ($2f+1 / f+1$) niti na centralni bezbednosni argument algoritma, koji su u potpunosti implementirani i verifikovani kroz demonstracione scenarije opisane u narednom poglavlju.

6. Demonstracioni scenariji i verifikacija ispravnosti

Implementirana su četiri nezavisna demonstraciona scenarija, svaki sa sopstvenim skupom replika, mrežnim modelom i modelom grešaka

Scenario 1 – Normalan rad

Četiri replike ($N=4$, $f=1$), bez ikakvih grešaka. Klijent šalje jedan zahtev; zahtev prolazi kroz sve tri faze protokola u view-u 0, sve replike ga izvršavaju istim redosledom, a klijent prikuplja tražena $f+1=2$ podudarna odgovora.

```
=====
Practical Byzantine Fault Tolerance (PBFT)
Based on: Castro & Liskov, ACM TOCS 20(4), 2002
Runtime: goroutines + channels (week03_04_parallel)
=====
```

```
=====
EXAMPLE 1: NORMAL CASE – 4 replicas (f=1), no faults
Reference: Castro & Liskov, Section 4.3 (Fig. 1)
=====
```

```
>>> Starting PARALLEL simulation (max 10s, idle timeout 700ms)
```

```
[client-1] Sending SET x=1(client=client-1,ts=1) to all 4 replicas
[R3] Received REQUEST SET x=1(client=client-1,ts=1) (view=0, primary=R0)
[R0] Received REQUEST SET x=1(client=client-1,ts=1) (view=0, primary=R0)
[R1] Received REQUEST SET x=1(client=client-1,ts=1) (view=0, primary=R0)
[R2] Received REQUEST SET x=1(client=client-1,ts=1) (view=0, primary=R0)
[R2] Accepted PRE-PREPARE(v=0,n=1,d=71fbdc22947c) from primary R0
[R3] Accepted PRE-PREPARE(v=0,n=1,d=71fbdc22947c) from primary R0
[R1] Accepted PRE-PREPARE(v=0,n=1,d=71fbdc22947c) from primary R0
[R3] PREPARED seq=1 (d=71fbdc22947c) with 3/4 votes – broadcasting COMMIT
[R1] PREPARED seq=1 (d=71fbdc22947c) with 3/4 votes – broadcasting COMMIT
[R2] PREPARED seq=1 (d=71fbdc22947c) with 3/4 votes – broadcasting COMMIT
[R0] PREPARED seq=1 (d=71fbdc22947c) with 3/4 votes – broadcasting COMMIT
[R2] *** EXECUTED seq=1 op=SET x=1 -> result=SET x=1#1 ***
[R3] *** EXECUTED seq=1 op=SET x=1 -> result=SET x=1#1 ***
[client-1] Received REPLY from R2: result="SET x=1#1" (1/2 matching)
[client-1] Received REPLY from R3: result="SET x=1#1" (2/2 matching)
[client-1] *** ACCEPTED result "SET x=1#1" – collected f+1=2 matching replies ***
[R0] *** EXECUTED seq=1 op=SET x=1 -> result=SET x=1#1 ***
[R1] *** EXECUTED seq=1 op=SET x=1 -> result=SET x=1#1 ***
[client-1] Received REPLY from R0: result="SET x=1#1" (3/2 matching)
[client-1] Received REPLY from R1: result="SET x=1#1" (4/2 matching)
```

```
>>> Simulation ended.
```

```
--- Execution Summary ---
Total events: 1925
SEND:      35
ENQUEUE:   35
DEQUEUE:   630
DELIVER:   35
DROP:      595
RETRANSMIT: 595
-----

--- PBFT Property Checks ---
[PASS] Agreement (total order): all replicas agree on the executed digest for every sequence number (1 checked)
[PASS] No double execution: every replica executed each sequence number at most once
[PASS] View progress: no view change was needed in this scenario
[PASS] Client quorum: client client-1 collected 4/4 matching replies ( $\geq f+1=2$ ) for result "SET x=1#1"

All properties: PASS
-----
```

Scenario 2 – Pad primary-a

Četiri replike; primary (R0) se gasi pre nego što uspe da obradi klijentski zahtev. Preostale tri replike, nakon isteka tajmera, sprovode promenu view-a; nova primary replika (R1, određena formulom $v \bmod N$) preuzima vodstvo i ponovo predlaže isti zahtev, koji zatim uspešno prolazi kroz sve faze i biva izvršen.

```
=====
EXAMPLE 2: PRIMARY CRASH – view change (4 replicas, f=1)
R0 (primary of view 0) crashes before ordering the request.
Reference: Castro & Liskov, Section 4.5
=====
```

```
>>> Crashing primary R0 before it can order anything <<<
```

```
>>> Starting PARALLEL simulation (max 15s, idle timeout 1.2s)
```

```
[client-1] Sending SET x=2(client=client-1,ts=1) to all 4 replicas
[R1] Received REQUEST SET x=2(client=client-1,ts=1) (view=0, primary=R0)
[R3] Received REQUEST SET x=2(client=client-1,ts=1) (view=0, primary=R0)
[R2] Received REQUEST SET x=2(client=client-1,ts=1) (view=0, primary=R0)
[R2] *** TIMEOUT in view=0 – suspecting primary R0, initiating VIEW-CHANGE to view=1 ***
[R1] *** TIMEOUT in view=0 – suspecting primary R0, initiating VIEW-CHANGE to view=1 ***
[R3] *** TIMEOUT in view=0 – suspecting primary R0, initiating VIEW-CHANGE to view=1 ***
[R1] *** Collected 2f+1 VIEW-CHANGE votes for view=1 – becoming primary, sending NEW-VIEW ***
[R1] Re-proposing pending request SET x=2(client=client-1,ts=1) as seq=1 in new view=1
[R3] Accepted NEW-VIEW for view=1 (new primary=R1)
[R3] Accepted PRE-PREPARE(v=1,n=1,d=87bcfaa30340) from primary R1
[R2] Accepted NEW-VIEW for view=1 (new primary=R1)
[R2] Accepted PRE-PREPARE(v=1,n=1,d=87bcfaa30340) from primary R1
[R2] PREPARED seq=1 (d=87bcfaa30340) with 3/4 votes – broadcasting COMMIT
[R3] PREPARED seq=1 (d=87bcfaa30340) with 3/4 votes – broadcasting COMMIT
[R1] PREPARED seq=1 (d=87bcfaa30340) with 3/4 votes – broadcasting COMMIT
[R1] *** EXECUTED seq=1 op=SET x=2 -> result=SET x=2#1 ***
[R2] *** EXECUTED seq=1 op=SET x=2 -> result=SET x=2#1 ***
[R3] *** EXECUTED seq=1 op=SET x=2 -> result=SET x=2#1 ***
[client-1] Received REPLY from R1: result="SET x=2#1" (1/2 matching)
[client-1] Received REPLY from R2: result="SET x=2#1" (2/2 matching)
[client-1] *** ACCEPTED result "SET x=2#1" – collected f+1=2 matching replies ***
[client-1] Received REPLY from R3: result="SET x=2#1" (3/2 matching)
```

```
>>> Simulation ended.
```

```
--- Execution Summary ---
```

```
Total events: 3962
RETRANSMIT: 1257
CRASH: 1
DROP: 1272
SEND: 48
ENQUEUE: 48
DEQUEUE: 1304
DELIVER: 32
-----
```

```
--- PBFT Property Checks ---
```

```
[PASS] Agreement (total order): all replicas agree on the executed digest for every sequence number (1 checked)
[PASS] No double execution: every replica executed each sequence number at most once
[PASS] View progress: system advanced to view 1 after a view change was triggered
[PASS] Client quorum: client client-1 collected 3/3 matching replies (>= f+1=2) for result "SET x=2#1"
```

```
All properties: PASS
```

Scenario 3 – Byzantine (zlonamerno) ponašanje primary-a

Četiri replike; primary (R0) je konfigurisan da equivocira – šalje dve različite verzije istog zahteva dvema odvojenim grupama backup replika. Kako matematički nijedna od dve grupe ne može samostalno dostići jak kvorum od $2f+1$ glasova, obrada zahteva se zaglavljuje, replike prijavljuju sumnju putem tajmera, i sprovodi se promena view-a. Novi primary (R1) ponovo predlaže isti, nesporni zahtev, koji ovoga puta uspešno prolazi kroz sve faze – čime se potvrđuje da nijedna od dve prvobitno sukobljene verzije zahteva nikada nije izvršena ni na jednoj replici.

```
=====
EXAMPLE 3: BYZANTINE PRIMARY – equivocation (4 replicas, f=1)
R0 sends conflicting PRE-PREPAREs; view change recovers.
Reference: Castro & Liskov, Section 4.3 (prepared certificate)
=====

>>> Starting PARALLEL simulation (max 15s, idle timeout 1.2s)

[client-1] Sending SET x=3(client=client-1,ts=1) to all 4 replicas
[R0] Received REQUEST SET x=3(client=client-1,ts=1) (view=0, primary=R0)
[R2] Received REQUEST SET x=3(client=client-1,ts=1) (view=0, primary=R0)
[R1] Received REQUEST SET x=3(client=client-1,ts=1) (view=0, primary=R0)
[R0] *** BYZANTINE PRIMARY: equivocating on seq=1 – [R1] get digest=fec330a7f421, [R2 R3] get digest=24cf21b885ec ***
[R3] Received REQUEST SET x=3(client=client-1,ts=1) (view=0, primary=R0)
[R3] Accepted PRE-PREPARE(v=0,n=1,d=24cf21b885ec) from primary R0
[R2] Accepted PRE-PREPARE(v=0,n=1,d=24cf21b885ec) from primary R0
[R1] Accepted PRE-PREPARE(v=0,n=1,d=fec330a7f421) from primary R0
[R1] *** TIMEOUT in view=0 – suspecting primary R0, initiating VIEW-CHANGE to view=1 ***
[R3] *** TIMEOUT in view=0 – suspecting primary R0, initiating VIEW-CHANGE to view=1 ***
[R2] *** TIMEOUT in view=0 – suspecting primary R0, initiating VIEW-CHANGE to view=1 ***
[R0] *** TIMEOUT in view=0 – suspecting primary R0, initiating VIEW-CHANGE to view=1 ***
[R1] *** Collected 2f+1 VIEW-CHANGE votes for view=1 – becoming primary, sending NEW-VIEW ***
[R1] Re-proposing pending request SET x=3(client=client-1,ts=1) as seq=1 in new view=1
[R0] Accepted NEW-VIEW for view=1 (new primary=R1)
[R3] Accepted NEW-VIEW for view=1 (new primary=R1)
[R3] Accepted PRE-PREPARE(v=1,n=1,d=fec330a7f421) from primary R1
[R0] Accepted PRE-PREPARE(v=1,n=1,d=fec330a7f421) from primary R1
[R2] Accepted NEW-VIEW for view=1 (new primary=R1)
[R2] Accepted PRE-PREPARE(v=1,n=1,d=fec330a7f421) from primary R1
[R1] PREPARED seq=1 (d=fec330a7f421) with 3/4 votes – broadcasting COMMIT
[R3] PREPARED seq=1 (d=fec330a7f421) with 3/4 votes – broadcasting COMMIT
[R0] PREPARED seq=1 (d=fec330a7f421) with 3/4 votes – broadcasting COMMIT
[R2] PREPARED seq=1 (d=fec330a7f421) with 3/4 votes – broadcasting COMMIT
[R0] *** EXECUTED seq=1 op=SET x=3 -> result=SET x=3#1 ***
[R1] *** EXECUTED seq=1 op=SET x=3 -> result=SET x=3#1 ***
[R3] *** EXECUTED seq=1 op=SET x=3 -> result=SET x=3#1 ***
[R2] *** EXECUTED seq=1 op=SET x=3 -> result=SET x=3#1 ***
[client-1] Received REPLY from R0: result="SET x=3#1" (1/2 matching)
[client-1] Received REPLY from R1: result="SET x=3#1" (2/2 matching)
[client-1] *** ACCEPTED result "SET x=3#1" – collected f+1=2 matching replies ***
[client-1] Received REPLY from R3: result="SET x=3#1" (3/2 matching)
```

```

[client-1] Received REPLY from R3: result="SET x=3#1" (3/2 matching)
[client-1] Received REPLY from R2: result="SET x=3#1" (4/2 matching)

>>> Simulation ended.

--- Execution Summary ---
Total events: 6138
SEND:        71
ENQUEUE:     71
DELIVER:     67
DROP:        1955
RETRANSMIT:  1952
DEQUEUE:     2022
-----

--- PBFT Property Checks ---
[PASS] Agreement (total order): all replicas agree on the executed digest for every sequence number (1 checked)
[PASS] No double execution: every replica executed each sequence number at most once
[PASS] View progress: system advanced to view 1 after a view change was triggered
[PASS] Client quorum: client client-1 collected 4/4 matching replies ( $\geq f+1=2$ ) for result "SET x=3#1"

All properties: PASS
-----

```

Scenario 4 – Gubitak poruka kod jedne replike

Sedam replika ($N=7$, $f=2$); jedna replika (R6) prima manje od jedan odsto poruka koje su joj upućene, simulirajući ozbiljno degradiranu mrežnu vezu. Preostalih šest replika, koje čine dovoljan kvorum od $2f+1=5$, uspešno obrađuju zahtev i klijent prihvata rezultat, bez ikakve potrebe za promenom view-a – demonstrirajući da algoritam toleriše potpunu neaktivnost do f replika bez ugrožavanja ispravnosti ili dostupnosti sistema.

```
=====
EXAMPLE 4: SLOW BACKUP – message omission (7 replicas, f=2)
R6's messages are dropped 90% of the time; no view change needed.
Reference: Castro & Liskov, Section 2 (fair-loss / omission)
=====
```

```
>>> Starting PARALLEL simulation (max 15s, idle timeout 1.2s)
```

```
[client-1] Sending SET x=4(client=client-1,ts=1) to all 7 replicas
[R0] Received REQUEST SET x=4(client=client-1,ts=1) (view=0, primary=R0)
[R2] Received REQUEST SET x=4(client=client-1,ts=1) (view=0, primary=R0)
[R1] Received REQUEST SET x=4(client=client-1,ts=1) (view=0, primary=R0)
[R4] Received REQUEST SET x=4(client=client-1,ts=1) (view=0, primary=R0)
[R4] Accepted PRE-PREPARE(v=0,n=1,d=ff26af853e02) from primary R0
[R5] Received REQUEST SET x=4(client=client-1,ts=1) (view=0, primary=R0)
[R2] Accepted PRE-PREPARE(v=0,n=1,d=ff26af853e02) from primary R0
[R1] Accepted PRE-PREPARE(v=0,n=1,d=ff26af853e02) from primary R0
[R3] Received REQUEST SET x=4(client=client-1,ts=1) (view=0, primary=R0)
[R5] Accepted PRE-PREPARE(v=0,n=1,d=ff26af853e02) from primary R0
[R3] Accepted PRE-PREPARE(v=0,n=1,d=ff26af853e02) from primary R0
[R1] PREPARED seq=1 (d=ff26af853e02) with 5/7 votes – broadcasting COMMIT
[R0] PREPARED seq=1 (d=ff26af853e02) with 5/7 votes – broadcasting COMMIT
[R2] PREPARED seq=1 (d=ff26af853e02) with 5/7 votes – broadcasting COMMIT
[R3] PREPARED seq=1 (d=ff26af853e02) with 5/7 votes – broadcasting COMMIT
[R5] PREPARED seq=1 (d=ff26af853e02) with 5/7 votes – broadcasting COMMIT
[R4] PREPARED seq=1 (d=ff26af853e02) with 5/7 votes – broadcasting COMMIT
[R5] *** EXECUTED seq=1 op=SET x=4 -> result=SET x=4#1 ***
[R1] *** EXECUTED seq=1 op=SET x=4 -> result=SET x=4#1 ***
[R2] *** EXECUTED seq=1 op=SET x=4 -> result=SET x=4#1 ***
[R0] *** EXECUTED seq=1 op=SET x=4 -> result=SET x=4#1 ***
[R4] *** EXECUTED seq=1 op=SET x=4 -> result=SET x=4#1 ***
[R3] *** EXECUTED seq=1 op=SET x=4 -> result=SET x=4#1 ***
[client-1] Received REPLY from R5: result="SET x=4#1" (1/3 matching)
[client-1] Received REPLY from R1: result="SET x=4#1" (2/3 matching)
[client-1] Received REPLY from R2: result="SET x=4#1" (3/3 matching)
[client-1] *** ACCEPTED result "SET x=4#1" – collected f+1=3 matching replies ***
[client-1] Received REPLY from R0: result="SET x=4#1" (4/3 matching)
[client-1] Received REPLY from R4: result="SET x=4#1" (5/3 matching)
[client-1] Received REPLY from R3: result="SET x=4#1" (6/3 matching)
[R6] Accepted PRE-PREPARE(v=0,n=1,d=ff26af853e02) from primary R0
[R6] Received REQUEST SET x=4(client=client-1,ts=1) (view=0, primary=R0)
```

```

[R6] Received REQUEST SET x=4(client=client-1,ts=1) (view=0, primary=R0)
[R6] PREPARED seq=1 (d=ff26af853e02) with 5/7 votes – broadcasting COMMIT
[R6] *** EXECUTED seq=1 op=SET x=4 -> result=SET x=4#1 ***
[client-1] Received REPLY from R6: result="SET x=4#1" (7/3 matching)

>>> Simulation ended.

--- Execution Summary ---
Total events: 20440
DEQUEUE:      6778
DROP:         6674
RETRANSMIT:   6676
SEND:         104
ENQUEUE:      104
DELIVER:      104
-----

--- PBFT Property Checks ---
[PASS] Agreement (total order): all replicas agree on the executed digest for every sequence number (1 checked)
[PASS] No double execution: every replica executed each sequence number at most once
[PASS] View progress: no view change was needed in this scenario
[PASS] Client quorum: client client-1 collected 7/7 matching replies (>= f+1=3) for result "SET x=4#1"

All properties: PASS
-----

```

Za svaki scenario, nakon završetka simulacije, automatski se proveravaju sledeća svojstva nad zabeleženim tokom izvršavanja:

- Saglasnost (Agreement) – da li su se sve replike, koje su izvršile zahtev sa istim rednim brojem, složile oko identičnog sadržaja tog zahteva (osnovno svojstvo bezbednosti, odnosno oblik linearizabilnosti definisan u Dodatku B originalnog rada).
- Odsustvo dvostrukog izvršavanja – da nijedna replika nije izvršila isti redni broj više od jednom.
- Napredak promene view-a – ukoliko je promena view-a bila neophodna, da li je sistem zaista i uspešno prešao u viši view (svojstvo živosti).
- Klijentski kvorum – da li je klijent zaista prikupio najmanje $f+1$ podudarnih odgovora pre prihvatanja rezultata.

U svim izvršenim scenarijima, sve četiri provere prolaze uspešno, čime je potvrđeno da implementacija ispravno demonstrira ključna svojstva algoritma PBFT – bezbednost koja se nikada ne narušava, čak ni pod aktivnim, zlonamernim napadom, i živost koja se, uz konačno kašnjenje mreže, uvek na kraju uspostavlja.

7. Zaključak

Algoritam PBFT predstavlja značajan pomak u oblasti otpornosti na vizantijske greške, jer je prvi put pokazao da je moguće postići i praktičnu efikasnost i bezbednost u potpuno asinhronim mrežama, bez oslanjanja na nerealne pretpostavke o sinhronosti. Ključna ideja algoritma –

kombinacija rotirajućeg vodstva (primary-backup pristup) i mehanizma kvoruma ($2f+1$) – omogućava da sistem tačno razlikuje dva odvojena svojstva: bezbednost, koja mora važiti bezuslovno, u svakom trenutku, bez obzira na ponašanje mreže ili neispravnih čvorova; i živost, koja zavisi od konačnog kašnjenja mreže, ali čije privremeno izostajanje nikada ne dovodi u pitanje ispravnost već izvršenih operacija.

Kroz implementaciju simulatora u programskom jeziku Go, ovaj rad je pokazao da se osnovna logika algoritma – tri faze normalnog rada (pre-prepare, prepare, commit) i mehanizam promene view-a – može verno predstaviti kroz konkurentne, nezavisne izvršne jedinice koje komuniciraju isključivo razmenom poruka, u skladu sa modelom stvarnih distribuiranih sistema. Četiri demonstraciona scenarija – normalan rad, par primary-a, zlonamerno ponašanje primary-a i gubitak poruka kod jedne replike – obuhvataju sve glavne kategorije grešaka predviđene specifikacijom projekta i, kroz automatizovane provere ispravnosti, potvrđuju da implementacija zadovoljava teorijske garancije algoritma u granicama $N = 3f+1$.

Kao pravac daljeg razvoja, implementacija bi se mogla proširiti mehanizmima koji nisu obuhvaćeni ovim radom: periodičnim čuvanjem stanja i sabiranjem otpadaka (checkpoints i garbage collection), potpunim prenosom stanja između replika (state transfer), kao i proaktivnim oporavkom kompromitovanih replika (proactive recovery) – mehanizmom koji autori originalnog rada posebno ističu kao ključan za dugotrajnu bezbednost sistema, jer omogućava toleranciju neograničenog broja grešaka tokom životnog veka sistema, sve se broj istovremeno neispravnih replika u svakom trenutku održava ispod granice od jedne trećine.